



field guide · seeing with radio · 6 projects · jun 2026

> build six things that see with radio_

Six EE Projects That See With Radio

Radar, micro-Doppler, WiFi sensing, and passive detection – six builds that read the world by how it reflects and shifts radio. From a \$12 chip that senses motion through a wall to tracking aircraft with no transmitter of your own. Cheap hardware; real signal processing.

01	Doppler Motion Radar	Wave your hand and watch its speed appear – the Doppler core inside every radar	\$20
02	Radar Speed Gun	Calibrate that radar into real mph – clock a passing car or a fastball	\$30
03	FMCW Ranging Radar	Stop measuring speed, start measuring distance – the principle inside adaptive cruise control	\$70
04	Micro-Doppler Vital-Signs Radar	Read breathing and a heartbeat across the room – contactless, through clothing	\$30
05	WiFi-Sensing Motion Detector	Sense motion through a wall with a \$10 chip and no camera	\$12
06	Passive Radar	Track aircraft using a transmitter you don't own – bistatic radar with zero emissions	\$70

● By Enes · @1nska · Electrical Engineer – a Build List field guide

Doppler Motion Radar

Wave your hand and watch its speed appear – the Doppler core inside every radar

\$20

beginner

1 weekend

why this matters

Every radar on Earth starts with one idea: a moving object shifts the frequency of the radio it reflects. A pocket microwave module like the HB100 transmits a steady ~10.5 GHz tone and mixes it with the echo, and what comes out is an audio-frequency “beat” whose pitch is directly proportional to the target’s speed. Amplify that beat, sample it on an Arduino, run an FFT, and you can read how fast your hand – or a passing person – is moving, in real time. It is the cheapest possible way to build genuine radar, the payoff is immediate and visual, and it is the foundation every later project in this guide builds on.

WHAT THIS PROVES TO A RECRUITER

You understand the Doppler effect, continuous-wave radar, mixing down to baseband, low-noise audio amplification, and basic FFT-based spectral estimation on a microcontroller – the literal core of radar signal processing.

→ **the job this maps to:** radar & sensing engineering – automotive, security / occupancy sensing, and defense radar groups.

how it works

- ▶ **Continuous-wave Doppler:** The HB100 transmits an unmodulated ~10.525 GHz carrier and mixes the returning echo with it. A stationary target returns the same frequency (zero beat); a moving target returns a Doppler-shifted frequency, and the mixer outputs the difference – a low audio tone.
- ▶ **The Doppler equation:** The beat frequency is $f_d = 2 \cdot v \cdot f_0 / c$. At 10.5 GHz, every 1 m/s of radial speed produces ~70 Hz of beat, so walking speed (~1.5 m/s) lands near 100 Hz – squarely in the audio band, which is why a cheap op-amp can handle it.
- ▶ **Amplifying microvolts:** The raw IF output is tiny (microvolts to millivolts) and low-frequency. A two-stage op-amp with band-pass shaping (roughly 20 Hz–3 kHz) lifts it to a clean line level the ADC can read, while rejecting DC offset and mains hum.
- ▶ **Sampling and the FFT:** An Arduino or ESP32 samples the amplified beat at a few kHz and runs an FFT. The peak bin is the dominant Doppler frequency; convert it back to m/s with the equation and you have a live speed readout.
- ▶ **Reading direction (bonus):** A plain HB100 cannot tell approaching from receding (both shift the magnitude up). Modules with I/Q outputs, or an added quadrature mixer, recover the sign of the shift – the same trick higher-end Doppler sensors use.

key design decisions

- **Why continuous-wave instead of pulsed?** CW Doppler is the simplest radar to build: no fast pulse timing, no high-speed sampling, just a steady tone and an audio-rate mixer output. The tradeoff is that plain CW gives you speed but not range – which is exactly why project 3 adds a frequency sweep to recover distance.
- **Why does the signal land in the audio band?** The beat is the tiny difference between transmit and echo frequencies, and for everyday speeds that difference is only tens to hundreds of hertz. That is a gift: a \$0.50 op-amp and an 8-bit ADC are fast enough, because all the hard 10.5 GHz RF happens inside the module.
- **Why an FFT over counting zero-crossings?** Real returns contain multiple speeds and noise. Zero-crossing counting collapses that to one number and fails on weak or multi-target signals; an FFT shows the whole Doppler spectrum, so you can pick the true peak, see multiple movers, and set a noise threshold.

bill of materials

component	part / spec	qty	~cost
HB100 Doppler module	10.525 GHz CW microwave radar	1	\$4
Op-amp	LM358 or MCP6002 (dual)	1	\$1
Microcontroller	Arduino Uno / Nano or ESP32	1	\$8
Passives	resistors + caps for amp and filter	–	\$3
Breadboard + jumpers	prototyping	1	\$4

build steps

1 Power the module, find the IF pin

Give the HB100 a clean 5 V and locate its IF output. Wave your hand in front of it and scope (or listen to) the pin – you should see a faint low-frequency wiggle that tracks your motion. That confirms the module works before you build anything.

2 Build the amplifier

Wire a two-stage non-inverting op-amp with a gain of a few hundred and a band-pass shape (~ 20 Hz–3 kHz). AC-couple the input to block the module's DC offset. The output should swing cleanly when you move in front of the sensor.

3 Digitize the beat

Feed the amplified signal into your microcontroller's ADC, biased to mid-rail, and sample at ~ 4 –8 kHz. Print or plot the raw samples and confirm faster motion gives a higher-frequency wiggle.

4 Run the FFT, convert to speed

Use an FFT to find the peak bin, then apply $f_d = 2 \cdot v \cdot f_0 / c$ to get m/s and km/h. Show it on serial or a small screen. Wave slow then fast and watch the number rise.

5 Calibrate and threshold

Walk toward the sensor at a known pace (or roll a ball) to check the reading, and set a noise floor so an empty room reads zero. You now have a working speed-sensing radar – the exact front end project 2 turns into a speed gun.

interview questions

- What physically causes the frequency shift a Doppler radar measures?
- Derive the relationship between target speed and beat frequency for a CW radar.
- Why does the Doppler beat for everyday speeds fall in the audio band?
- What information does a plain CW radar lose, and how do you recover range?
- How does a CW Doppler sensor distinguish approaching from receding targets?
- Why is an FFT a better detector than zero-crossing counting for real returns?

Radar Speed Gun

Calibrate that radar into real mph – clock a passing car or a fastball

\$30

beginner

1 weekend

why this matters

A police speed gun and a baseball radar are, at heart, the project-1 Doppler front end with a calibrated readout and a display. This build locks down the speed calibration, adds peak-hold and a clean numeric display, and aims it at real targets: a car going past, a thrown ball, a bike. The result looks and behaves like a commercial speed gun – but you built the signal chain yourself and can explain every number on the screen. It is the most immediately impressive, demo-friendly upgrade in the guide, and a perfect “before / after” with project 1 for content.

WHAT THIS PROVES TO A RECRUITER

You can take a raw sensor and turn it into a calibrated instrument – fixed-point conversion, peak detection and hold, display integration, and validating a measurement against a known reference. That instrumentation discipline is exactly what test-and-measurement and sensor teams hire for.

→ **the job this maps to:** measurement & instrumentation, test engineering, and embedded sensor roles – sporting-tech, automotive test, and T&M companies.

how it works

- ▶ **From beat frequency to mph:** The conversion is fixed: $\text{speed} = f_d \cdot c / (2 \cdot f_0)$. Precompute the constant for your module’s exact frequency so each FFT peak maps directly to km/h or mph with one multiply – fast enough to update several times a second.
- ▶ **Peak detection and hold:** A passing target’s speed peaks then fades as the geometry changes. The firmware tracks the maximum valid reading over a short window and “holds” it on the display – exactly how a real speed gun reports the fastest moment of a pitch or a car.
- ▶ **Cosine (angle) error:** Doppler measures only the velocity component along the beam. Aim off-axis by angle θ and the reading is low by $\cos \theta$. Aiming nearly head-on keeps the error tiny – and explains why speed traps are aimed straight down the road.
- ▶ **Antenna gain and range:** The bare module sees a few metres. Putting it at the focus of a cheap parabolic or horn (or even a metal can) narrows the beam and adds gain, extending useful range and rejecting off-axis clutter – a tangible lesson in antenna directivity.
- ▶ **Display and packaging:** An OLED or 7-segment shows the held speed; a momentary button re-arms it. Mounting the module, amp, MCU, and display in a 3D-printed grip turns the breadboard into something that reads as a real device on camera.

bill of materials

component	part / spec	qty	~cost
Project 1 front end	HB100 + amp + microcontroller	1	\$20
Display	SSD1306 OLED or 7-segment	1	\$5
Reflector (optional)	parabolic / horn or metal can	1	\$3
Enclosure	3D-printed grip / project box	1	\$4
Button + misc	re-arm, wiring	—	\$2

build steps

1 Lock the calibration constant

Measure or look up your module's exact transmit frequency, compute $\text{speed} = f_d \cdot c / (2 \cdot f_0)$, and bake the constant into firmware. Verify on a known mover before trusting it.

2 Add peak-hold logic

Track the maximum valid speed over a ~1-2 s window, ignore sub-threshold noise, and freeze the result until a button re-arms it. Test by walking quickly past the beam.

3 Validate against a reference

Check readings against a phone GPS speedometer in a car (as a passenger, safely), or a second method for a thrown ball. Aim nearly head-on to minimise cosine error and confirm the numbers track.

4 Add a reflector for range

Mount the module at the focus of a parabolic / horn or inside a metal can to narrow the beam. Note how range improves and off-axis clutter drops – and how aiming becomes more critical.

5 Package it

Fit everything into a 3D-printed grip or box with the display on top and a trigger / re-arm button. Now film it clocking cars or pitches – a clean, legible speed gun you built end to end.

interview questions

- Write the exact conversion from Doppler beat frequency to vehicle speed.
- What is cosine error in a speed radar, and how do you minimise it?
- Why and how does a speed gun “hold” a peak reading?
- How does adding a horn or dish change the measurement, beyond range?
- How would you validate your speed gun's accuracy against a trusted reference?
- What error sources set the ultimate accuracy of your reading?

FMCW Ranging Radar

Stop measuring speed, start measuring distance – the principle inside adaptive cruise control

\$70

advanced

1–2 weekends

why this matters

Doppler tells you how fast something moves; FMCW (frequency-modulated continuous-wave) radar tells you how far away it is. By sweeping the transmit frequency up in a linear “chirp,” the returning echo is delayed – and because the transmitter has moved on in frequency during that delay, mixing the two produces a beat whose frequency encodes range. This is the exact technique inside automotive adaptive-cruise and weather radar, and building even a short-range version is the leap from “I detected motion” to “I measured the scene.” It is the most ambitious build here, so the trick is to build it module-based or as the classic coffee-can radar rather than laying out microwave boards from scratch.

WHAT THIS PROVES TO A RECRUITER

You understand FMCW theory, chirp generation and linearity, range-from-beat-frequency, and the FFT processing (range FFT, and range-Doppler if you go further) that underpins modern radar – the core skill set of automotive and imaging-radar engineering.

→ **the job this maps to:** automotive radar and imaging-radar engineering – Tier-1 automotive suppliers, mmWave chip vendors (TI, Infineon, NXP), and sensing startups.

how it works

- ▶ **The chirp:** Instead of a fixed tone, FMCW sweeps the carrier linearly across a bandwidth B over a time T . The wider the bandwidth, the finer the range resolution: $\Delta r = c/(2B)$. A few hundred MHz of sweep gives sub-metre resolution.
- ▶ **Range from beat frequency:** An echo arrives delayed by $\tau = 2R/c$. During that delay the transmit frequency has ramped on, so mixing transmit with echo gives a beat proportional to range: $R = c \cdot f_b \cdot T / (2B)$. Closer targets give a lower beat, farther ones a higher beat.
- ▶ **Generating a linear ramp:** A VCO or a chirp-capable source (e.g. an ADF4158-driven module) produces the sweep. Linearity matters: a non-linear ramp smears the range bins, so a PLL or calibration keeps the sweep straight.
- ▶ **The coffee-can architecture:** The classic MIT short-course radar uses two coffee cans as transmit / receive horns at ~ 2.4 GHz, a cheap mixer, and an audio interface or SDR to capture the beat. It sidesteps custom microwave layout while teaching the entire FMCW chain.
- ▶ **Range FFT and plotting:** FFT one chirp's beat signal and each peak's frequency maps to a target range – a 1-D range profile. Stack many chirps and add a second FFT for Doppler, separating targets by speed as well: a range-Doppler map.

BUILD NOTE

Don't lay out 24 GHz from scratch. Use a chirp-capable module, or build the MIT "coffee-can" radar at 2.4 GHz – the cantennas and an \$8 mixer do the hard RF, so you focus on the chirp and the FFT.

bill of materials

component	part / spec	qty	~cost
24 GHz module or 2 cans	dev board (IPM-165 / BGT24) or coffee-can horns	1	\$25-50
Mixer (coffee-can)	passive ring mixer	1	\$8
Ramp / PLL source	ADF4158 / VCO + sweep, or built-in	1	\$12
Capture	RTL-SDR or audio interface	1	\$20
Antennas / feed	cantennas or module's patch array	1	\$0-8

build steps

1 Pick your architecture

Either buy a chirp-capable 24 GHz module (least RF pain) or build the coffee-can 2.4 GHz radar. The module gets results fastest; the coffee-can teaches more. Commit before buying parts.

2 Generate and verify the chirp

Drive the VCO / PLL (or enable the module's sweep) and confirm a clean linear ramp over your bandwidth on a spectrum view. Linearity here directly sets your range accuracy.

3 Mix and capture one chirp

Mix transmit with the received echo and record the beat for a single sweep with your SDR or audio interface. A nearby wall or a corner reflector should give a clear return.

4 Run the range FFT

FFT one chirp's beat and convert the peak frequency to range with $R = c \cdot f_b \cdot T / (2B)$. Move a reflector and watch the peak shift to the correct distance – your range profile.

5 Make a range plot (and go further)

Sweep continuously and plot range vs time as targets move. For the full picture, stack chirps and add a Doppler FFT for a range-Doppler map – the same output a car's radar produces.

interview questions

- What sets the range resolution of an FMCW radar, and why?
- Derive how target range maps to the beat frequency in an FMCW system.
- Why does chirp linearity matter, and how is it maintained?
- How do you extend a range-only FMCW radar to also measure velocity?
- What are the tradeoffs between sweep bandwidth, range, and resolution?
- Why is FMCW preferred over pulsed radar for short-range automotive sensing?

Micro-Doppler Vital-Signs Radar

Read breathing and a heartbeat across the room – contactless, through clothing

\$30

intermediate

1 weekend

why this matters

The same Doppler module that clocks a car can, pointed at a still person's chest, pick up the sub-millimetre motion of breathing and the even tinier motion of a heartbeat. These show up as a slow "micro-Doppler" wobble riding on the radar's phase; band-pass the signal into a respiration band ($\sim 0.1\text{--}0.5$ Hz) and a cardiac band ($\sim 0.8\text{--}2$ Hz) and you can pull out breaths-per-minute and a pulse without touching the subject. It is a genuinely striking demo – contactless vitals from a \$4 module – and it introduces the phase-based micro-Doppler processing used in gesture sensing, fall detection, and contactless health.

WHAT THIS PROVES TO A RECRUITER

You understand phase / quadrature processing, micro-Doppler signatures, narrow-band filtering to separate physiological rhythms, and the limits of motion-based sensing. This phase-domain RF skill set maps onto biomedical sensing, gesture / presence radar, and the mmWave presence-sensing products shipping today.

→ **the job this maps to:** biomedical and contactless-sensing engineering – medical-device R&D, gesture / presence radar teams, and health-tech startups.

! NOT A MEDICAL DEVICE

This is a demonstration of micro-Doppler sensing, not a medical instrument. It can show a breathing and pulse rhythm on a plot; it is not validated, not safe to rely on, and must never be used to make a health decision or to diagnose anyone. Treat it as physics you can see – not a monitor.

how it works

- ▶ **Micro-Doppler from tiny motion:** A chest moving even a millimetre changes the radar path length, which shifts the echo's phase. Breathing moves the chest ~4-12 mm; a heartbeat ~0.2-0.5 mm. Both appear as slow phase modulation on the returned signal.
- ▶ **Quadrature (I/Q) phase:** Reliable extraction needs I and Q channels so you track phase directly rather than amplitude. A module with I/Q outputs (or an added quadrature mixer) lets you compute the small chest displacement over time.
- ▶ **Separating breath from heartbeat:** Respiration sits around 0.1-0.5 Hz and cardiac around 0.8-2 Hz. Two band-pass filters split the combined motion into a respiration waveform and a much smaller cardiac waveform you can rate-count.
- ▶ **Why it is hard:** The heartbeat signal is tiny and easily buried by body motion and breathing harmonics. The subject must be still; averaging over several seconds and rejecting large motions is what makes the pulse emerge.
- ▶ **Reading the rates:** FFT or peak-interval analysis on each filtered band gives breaths-per-minute and beats-per-minute. Plotting the two waveforms live is the demo: a slow rise-and-fall for breathing, a faster ripple for the heart.

bill of materials

component	part / spec	qty	~cost
I/Q Doppler module	HB100 (basic) or CDM324 / I-Q 24 GHz sensor	1	\$4-15
Low-noise amp	dual op-amp, low offset	1	\$2
ADC / MCU	ESP32 or audio interface to PC	1	\$8
Passives + breadboard	filters, wiring	—	\$5

build steps

1 Get clean I/Q (or phase)

Use an I/Q-capable module, or add a quadrature mixer, so you can recover phase – not just amplitude. Verify by slowly moving a flat reflector and watching the phase track distance.

2 Amplify and sample slowly

The vitals band is sub-2 Hz, so low-frequency, low-noise gain matters. Sample at tens of Hz and log to a PC. Sit still in front of the sensor and capture a minute of data.

3 Filter into two bands

In Python, band-pass ~0.1–0.5 Hz for respiration and ~0.8–2 Hz for cardiac, and plot both. The respiration waveform appears immediately; the cardiac one needs a still subject and clean data.

4 Extract rates

Run an FFT or peak-interval analysis on each band to get breaths-per-minute and an estimated pulse. Compare breathing to a manual count and the pulse to a fingertip oximeter to see how close – and how fragile – it is.

5 Demo it responsibly

Show the live waveforms and rates on screen, clearly labelled as a sensing demo and not a medical reading. Test how stillness, distance, and clothing change the result.

interview questions

- How does a heartbeat or breath produce a measurable radar signal?
- Why is I/Q (phase) processing needed for micro-Doppler vital signs?
- What frequency bands separate respiration from cardiac motion?
- Why is the cardiac signal so much harder to extract than respiration?
- What are the failure modes and ethical limits of radar vital-sign sensing?
- How would you reject gross body motion while preserving the vitals signal?

WiFi-Sensing Motion Detector

Sense motion through a wall with a \$10 chip and no camera

\$12

intermediate

1 weekend

why this matters

You do not even need a radar transmitter to sense motion – the WiFi already in the room will do it. A standard ESP32 can expose its WiFi channel-state information (CSI): a fine-grained measurement of how the radio channel between two devices changes. When a person moves, they perturb the multipath, and the CSI fluctuates in a way you can detect – presence and motion sensing, through walls, with a \$10 microcontroller and no camera. It is the most accessible entry point into the very active research area of WiFi sensing (the basis of the new IEEE 802.11bf sensing standard), and it costs almost nothing.

WHAT THIS PROVES TO A RECRUITER

You understand channel-state information, multipath and how bodies perturb it, time-series feature extraction (variance, correlation), and the basics of RF sensing without a dedicated radar. This maps directly onto the WiFi-sensing and wireless-ML work now moving from research into products.

→ **the job this maps to:** wireless sensing and ML research / engineering – WiFi-sensing startups, presence-detection product teams, and academic RF-ML labs.

! SENSE YOUR OWN SPACE

WiFi sensing can detect people through walls – so keep it to your own space. Sensing or profiling people who have not consented is a real privacy violation (and increasingly regulated). Build it to understand the technique and monitor your own home, not to surveil anyone else.

how it works

- ▶ **What CSI is:** Every WiFi packet is received across many OFDM subcarriers; CSI is the per-subcarrier amplitude and phase – essentially a fine-grained snapshot of the radio channel, far richer than the single RSSI number most devices expose.
- ▶ **Why motion shows up:** Indoors, signals arrive via many reflected paths. A moving body changes those paths, so the CSI shifts over time even when transmitter and receiver are fixed. Stillness gives steady CSI; motion gives fluctuating CSI.
- ▶ **Getting CSI from an ESP32:** Special firmware (the ESP32 CSI toolkit) exposes the CSI for received packets over serial. With another device generating traffic (a ping flood), you get a continuous stream of channel snapshots.
- ▶ **Detecting motion:** The simplest detector watches the variance or rate-of-change of the CSI across subcarriers; it spikes when someone moves and settles when the room is empty. Thresholding that gives a robust presence / motion signal.
- ▶ **Toward classification:** With labelled data you can go further – distinguishing walking from a fall, counting people, or coarse room-occupancy – which is exactly where the research (and 802.11bf) is heading.

bill of materials

component	part / spec	qty	~cost
ESP32 dev board	any common ESP32 (CSI-capable)	1	\$8
Second device	phone / router / another ESP32 for traffic	1	\$0
USB cable	serial logging	1	\$2
Laptop	Python CSI logger + analysis	1	–

build steps

1 Flash CSI firmware

Load the ESP32 CSI-toolkit firmware so the board reports channel-state info for received packets. Confirm CSI lines stream over serial.

2 Generate steady traffic

Have another device send a regular packet stream (e.g. a ping every few ms) to the ESP32 so you get a continuous CSI rate. Verify packets and CSI arrive together.

3 Log and visualise

Capture CSI to your laptop and plot subcarrier amplitudes over time. With the room empty the traces are steady; walk through and watch them jump.

4 Build a motion detector

Compute a rolling variance (or correlation drop) across subcarriers and threshold it. Tune so an empty room reads “no motion” and a person walking reliably triggers – even from another room.

5 Push further (optional)

Collect labelled clips (empty, walking, sitting) and train a simple classifier, or test through-wall range. Document what the CSI can and cannot distinguish.

interview questions

- What is WiFi channel-state information, and how does it differ from RSSI?
- Why does human motion change the CSI with fixed transmitter and receiver?
- What simple features detect motion in a CSI time series?
- What are the privacy implications and limits of WiFi sensing?
- How would you extend motion detection to activity classification?
- What is 802.11bf, and why is WiFi sensing becoming standardised?

Passive Radar

Track aircraft using a transmitter you don't own – bistatic radar with zero emissions

\$70

advanced

2 weekends

why this matters

The most elegant build here transmits nothing at all. A powerful FM or digital-TV station already floods the sky with radio; passive (bistatic) radar listens to that signal directly on one receiver and to its echoes off passing aircraft on a second, then cross-correlates the two to reveal the planes' range and Doppler. You build a radar that is completely silent – undetectable, license-free, and powered entirely by transmitters that already exist. It is the hardest project in the set because it demands two coherent, clock-locked receivers and real cross-ambiguity processing, which is exactly why it is the standout portfolio piece.

WHAT THIS PROVES TO A RECRUITER

You understand bistatic radar geometry, coherent multi-channel reception and clock synchronisation, the cross-ambiguity function (range-Doppler correlation), and direct-signal cancellation. Coherent multi-receiver SDR and correlation processing are advanced, in-demand skills for defense, surveillance, and signals work.

→ the job this maps to: defense and surveillance radar, and advanced SDR / DSP engineering – radar primes, national labs, and signals-intelligence teams.

how it works

- ▶ **Bistatic geometry:** The transmitter (an FM / TV tower) and your receiver sit in different places, so an aircraft echo travels tower→plane→you. The extra path length over the direct tower→you path encodes the target's bistatic range; the changing path encodes bistatic Doppler.
- ▶ **Two coherent channels:** One antenna captures the strong direct signal (the “reference”); a second captures the weak echoes (the “surveillance” channel). Both SDRs must share one clock so their samples are phase-coherent – otherwise correlation fails.
- ▶ **The illuminator:** Broadband FM stations make great illuminators (strong, wideband, always on); digital TV (DVB-T) is even better where present. You pick a strong nearby station as your transmitter of opportunity.
- ▶ **Direct-signal cancellation:** The direct signal is enormously stronger than the echoes and will swamp them. Adaptive filtering subtracts the reference from the surveillance channel so the faint target returns survive.
- ▶ **The cross-ambiguity function:** Correlating the cleaned surveillance signal against delayed, Doppler-shifted copies of the reference produces a range-Doppler map; aircraft appear as peaks. This 2-D correlation is the heart of passive radar.

BUILD NOTE

The whole project lives or dies on coherence – two receivers sharing one clock. A KrakenSDR (five coherent channels) makes this far easier than hand-modding dongles; get the clock right and the rest is DSP.

bill of materials

component	part / spec	qty	~cost
2× coherent RTL-SDR	clock-locked pair (KrakenSDR or modded dongles)	1	\$40-100
2 antennas	FM / VHF, or UHF for DVB-T	2	\$10
Splitter / clock	shared reference clock	1	\$5
Coax + SMA	feedlines	—	\$6
Laptop	GNU Radio + passive-radar processing	1	—

build steps

1 Lock two receivers to one clock

Use a coherent SDR (e.g. KrakenSDR), or modify two RTL-SDRs to share a single oscillator. Verify the two channels are phase-coherent – this is the make-or-break step.

2 Aim reference and surveillance antennas

Point one antenna at a strong FM / TV transmitter (reference) and the other toward the sky / area you want to watch (surveillance). Choose a strong, steady illuminator.

3 Capture both channels

Record reference and surveillance simultaneously in GNU Radio. Confirm the direct signal dominates the reference and the surveillance channel is mostly that same signal plus noise.

4 Cancel the direct path

Apply adaptive cancellation to subtract the reference from the surveillance channel, suppressing the direct signal so weak echoes can emerge.

5 Compute the range-Doppler map

Run the cross-ambiguity function (existing passive-radar code helps) to correlate the cleaned echo against shifted reference copies. Aircraft show up as moving peaks – confirm against a live ADS-B map.

interview questions

- What is bistatic radar, and how does its geometry differ from monostatic?
- Why must the two receivers in passive radar be phase-coherent?
- What makes a good illuminator of opportunity, and why?
- Why is direct-signal cancellation necessary, and how is it done?
- What does the cross-ambiguity function compute, and why is it central?
- How would you confirm a detected target is a real aircraft?

> wrap

You don't need permission – or a transmitter – to see with radio.

Doppler, FMCW, micro-Doppler, WiFi CSI, and passive bistatic radar are five faces of one idea: information lives in how the world bends, reflects, and shifts radio. The hardware is cheap – a \$4 module, a \$10 chip, a pair of dongles. The skill is in the signal processing: the mixing, the FFTs, the phase, the correlation. That is exactly the part employers in automotive radar, sensing, and defense actually hire for.

// the skill ladder

Build them in order and each one hands you the technique the next one needs:

Doppler → FMCW ranging → micro-Doppler → WiFi CSI → passive bistatic

DO THIS

Build even one of these. Screenshot the live plot – a velocity trace, a range profile, a CSI spike, a range-Doppler map – log the data, and put it on GitHub with a clear README and an honest note on scope and ethics. One radar reel and one clean repo prove you can pull real information out of radio.

Full build notes, parts links, and the next field guide go out in **The Build List**. The people actually shipping these – and trading coffee-can and clock-locking tricks – are in **Ohm's Club**.

– Enes (@1nska) · Electrical Engineer